WEEK 4 · LESSON 23 OF 24 · HANDOUT

Deploying Deep Learning Responsibly

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=cswXUNtUfuI — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython: Train, Serve, Deploy — an NLP model with PyTorch and Docker — https://www.youtube.com/watch?v=BSDW8PWUVF0

🎓 **Go deeper:** EuroPython 2023: PyTorch 2.0 — why should you care? — https://www.youtube.com/watch?v=B_KXmhCffzQ

🎓 **Go deeper:** MIT 6.S191 (2024): Building AI Models in the Wild — https://www.youtube.com/watch?v=ZAGiinWiFsE

Your goal in this lesson: Save and reload a trained network, measure its size and speed, serve it behind a web API, and plan monitoring, privacy and fairness for a deployed deep-learning system.

From notebook to product

A model in a notebook helps nobody. To be useful it must be saved, loaded by an application (a website, a phone app, a USSD or WhatsApp service), answer fast enough, and be **watched** while people use it.

We deploy the digit reader from Week 2.

```
import time
import io
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(digits.data / 16.0, digits.target,
                                                    test_size=0.2, random_state=0,
stratify=digits.target)
Xtr, Xte = torch.tensor(X_train, dtype=torch.float32), torch.tensor(X_test,
dtype=torch.float32)
ytr, yte = torch.tensor(y_train), torch.tensor(y_test)

class DigitNet(nn.Module):
    def __init__(self, hidden=64):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(64, hidden), nn.ReLU(), nn.Linear(hidden, 10))

    def forward(self, x):
        return self.net(x)

torch.manual_seed(0)
model = DigitNet()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
for epoch in range(40):
    perm = torch.randperm(len(Xtr))
    for start in range(0, len(perm), 32):
        idx = perm[start:start + 32]
        opt.zero_grad()
        F.cross_entropy(model(Xtr[idx]), ytr[idx]).backward()
        opt.step()
model.eval()
with torch.no_grad():
    test_acc = (model(Xte).argmax(1) == yte).float().mean().item()
print("test accuracy:", round(test_acc, 3))
```

Save a checkpoint

Save the weights **plus** everything needed to use them correctly:

```
checkpoint = {
    "state_dict": model.state_dict(),
    "hidden": 64,
    "input_scaling": "divide pixels by 16",
    "classes": list(range(10)),
    "test_accuracy": round(test_acc, 3),
    "torch_version": torch.__version__,
}
torch.save(checkpoint, "digitnet_v1.pt")

loaded = torch.load("digitnet_v1.pt", weights_only=False)
model2 = DigitNet(loaded["hidden"])
model2.load_state_dict(loaded["state_dict"])
model2.eval()
print("same predictions:", torch.equal(model2(Xte).argmax(1), model(Xte).argmax(1)))
```

Only load checkpoint files you trust: `weights_only=False` allows a file to run code when opened. For files from others, keep the default `weights_only=True` and save only tensors and plain values.

Size and speed

```
buffer = io.BytesIO()
torch.save(model.state_dict(), buffer)
print("model size:", round(buffer.getbuffer().nbytes / 1024, 1), "KB")

one = Xte[:1]
with torch.inference_mode():
    start = time.perf_counter()
    for _ in range(1000):
        model(one)
    per_image = (time.perf_counter() - start) / 1000
    start = time.perf_counter()
    model(Xte)
    batch_time = time.perf_counter() - start
print(f"one image: {per_image * 1e6:.0f} microseconds;  {len(Xte)} images in one batch:
{batch_time * 1e3:.1f} ms")
```

This tiny model is about 21 KB and answers in microseconds. A ResNet-50 is about 100 MB; a large language model needs many gigabytes and powerful GPUs. Ways to make big models practical:

- **Batching** — answer many requests together (much faster per item).
- **Half precision** (`model.half()` on a GPU) — half the memory, often faster.
- **Quantisation** — store weights in 8 bits or fewer; small models for phones.
- **Distillation** — train a small "student" to copy a big "teacher".
- **Export** to **ONNX**, TensorFlow Lite or Core ML to run on phones and edge devices without Python.

Serve it behind an API

A web API lets any app send an image and get an answer. Here with **FastAPI** — the same framework that powers the WAMUGII website:

```
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field

app = FastAPI(title="Digit reader")

class DigitImage(BaseModel):
    pixels: list[float] = Field(min_length=64, max_length=64)   # 8 x 8 values, 0..16

@app.post("/predict")
def predict(image: DigitImage):
    x = torch.tensor([image.pixels], dtype=torch.float32) / 16.0
    with torch.inference_mode():
        probs = F.softmax(model2(x), dim=1)[0]
    best = int(probs.argmax())
    confident = float(probs[best]) >= 0.8
    return {"digit": best, "confidence": round(float(probs[best]), 3),
            "action": "accept" if confident else "send to a person"}

client = TestClient(app)                 # test the API without starting a server
response = client.post("/predict", json={"pixels": digits.data[0].tolist()})
print(response.status_code, response.json())
print(client.post("/predict", json={"pixels": [1.0, 2.0]}).status_code)   # bad input is
rejected
```

In production you would run it with `uvicorn`, inside Docker, on a server — as you did for the WAMUGII platform. Notice two good habits: **input validation** (exactly 64 pixels) and a **confidence rule** that sends unsure cases to a person.

Watch it after launch

- **Inputs:** are new images darker, blurrier, from new phones? (data drift)
- **Outputs:** has the share of "send to a person" jumped?
- **Truth:** collect a sample of real cases with correct labels every month and re-measure accuracy.
- **Groups:** check results by region, language, device or skin tone where relevant.
- **Retrain** with new, representative data when performance falls — and test the new model before replacing the old one.

Responsible deep learning

- **Privacy:** photos, voices and messages are personal data. Rwanda's data protection law (Law No. 058/2021) requires a lawful purpose, consent or another legal basis, security, and limits on keeping data. Collect only what you need.
- **Consent and fairness to data givers:** farmers, patients and customers who provide data should know how it is used.
- **Transparency:** write a model card (Machine Learning Fundamentals, Lesson 23): purpose, training data, accuracy, limits.
- **Energy and cost:** training big models uses a lot of electricity; reuse pretrained models and choose the smallest model that does the job.
- **Human oversight:** high-stakes decisions (health, credit, justice) need a person in the loop.

Key terms

- **Checkpoint** — a saved file with weights and the settings needed to use them.
- **Inference** — using a trained model to make predictions.

- **Latency** — time to answer one request.
- **Quantisation / distillation** — making models smaller and faster.
- **API** — a web interface that apps call to get predictions.
- **Data drift** — real inputs changing over time.

Remember this: Deploying a network means saving it with its settings, serving it fast behind a validated API, routing unsure cases to people, and monitoring it — while protecting privacy and staying transparent.

Name: Date:

Activity

Task 1

Add `"version": "v1"` to the API response and a GET endpoint `/health` that returns `{"status": "ok"}`. Test both with `TestClient`.

Task 2

Time the model on batches of 1, 32 and 360 images. How does the time per image change?

Task 3

Make the API reject pixel values outside 0–16 (hint: `Field(ge=0, le=16)` on the items).

Task 4

Write a half-page monitoring plan for a crop-disease app used by 10,000 farmers.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What should a checkpoint contain besides the weights?

   A. Passwords
   B. The training data
   C. Nothing else
   D. The settings needed to rebuild and use the model correctly, such as input scaling

2. Why only load checkpoint files you trust?

   A. They contain labels
   B. They are large
   C. Loading with weights_only=False can run code from the file
   D. They expire

3. What does batching do for inference?

   A. Makes the model bigger
   B. Answers many requests together, much faster per item
   C. Changes the labels
   D. Stops monitoring

4. Which technique stores weights in 8 bits or fewer to make models smaller?

   A. Attention
   B. Dropout
   C. Quantisation
   D. Augmentation

5. Why does the API send low-confidence answers to a person?

   A. Because FastAPI requires it
   B. To slow the system down
   C. To collect passwords
   D. To keep a human in the loop where the model is unsure

6. Which Rwandan law governs personal data such as photos and voices?

   A. Law No. 058/2021 on the protection of personal data and privacy
   B. A traffic law
   C. There is none
   D. The tax code

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.